PROJETO DE ANÁLISE E FLUXO DE DADOS - OLAP E ETL

Carlos Alexandre Andrade Christo
Henrique Bruneli Santos
Victor Alexandre Vicente Pereira Turial
Ricardo Gomes Pereira
James Alves da Silva Junior

RESUMO

Este trabalho apresenta o desenvolvimento de um Data Warehouse para a área de vendas, baseado no banco de dados AdventureWorks da Microsoft, utilizando modelagem multidimensional no padrão Star Schema com uma tabela fato e seis dimensões, implementado no SQL Server. Foram definidos dez indicadores-chave de desempenho (KPIs) abrangendo receita, margem, sazonalidade e segmentação de clientes. O pipeline de ETL incremental, desenvolvido em Python, processa apenas registros novos ou alterados a cada execução. Os resultados mostram que o modelo suporta todos os indicadores definidos, validados por consultas SQL.

Palavras-chave: Data Warehouse; Star Schema; ETL Incremental; KPIs; AdventureWorks; SQL Server.

ABSTRACT

This work presents the development of a Data Warehouse for the sales area, based on Microsoft's AdventureWorks database, using multidimensional modeling in the Star Schema pattern with one fact table and six dimensions, implemented in SQL Server. Ten key performance indicators (KPIs) were defined, covering revenue, margin, seasonality and customer segmentation. The incremental ETL pipeline, developed in Python, processes only new or modified records at each execution. The results show that the model supports all defined indicators, validated through SQL queries.

Keywords: Data Warehouse; Star Schema; Incremental ETL; KPIs; AdventureWorks; SQL Server.

1 INTRODUÇÃO

A crescente geração de dados nas organizações criou a necessidade de ferramentas capazes de transformar grandes volumes de informação operacional em conhecimento estratégico. Nesse contexto, o Data Warehouse (DW) consolida-se como uma das principais arquiteturas para suporte à decisão, sendo amplamente adotado por empresas que buscam análises confiáveis e consistentes sobre seus processos de negócio (Kimball; Ross, 2013). Diferentemente dos bancos de dados transacionais (OLTP), projetados para operações de escrita frequentes e transações atômicas, os DWs são otimizados para consultas analíticas complexas, geralmente estruturados com modelagem multidimensional (Moraes; Silva; Antunes, 2024).

O presente trabalho tem por objetivo desenvolver um Data Warehouse voltado à área de vendas, utilizando como fonte o banco de dados AdventureWorks2016, amostra da Microsoft que simula uma empresa fictícia de fabricação e distribuição de bicicletas. A escolha se justifica pela riqueza estrutural do conjunto, que abrange domínios de vendas, produção, compras e recursos humanos, permitindo a construção de um modelo analítico robusto (Microsoft, 2026).

O escopo do trabalho abrange a análise do modelo OLTP do AdventureWorks, a proposição de um modelo estrela com seis dimensões e uma tabela fato, a definição de dez KPIs de vendas, a implementação do DW em SQL Server, o desenvolvimento de uma ETL incremental em Python e a validação dos indicadores por meio de consultas SQL analíticas.

2 REFERENCIAL TEÓRICO

2.1 MODELAGEM MULTIDIMENSIONAL

A modelagem multidimensional é uma técnica de design de banco de dados orientada a análises, proposta por Ralph Kimball na década de 1990, que organiza os dados em torno de dois tipos de tabelas: fatos e dimensões. As tabelas de fato armazenam métricas numéricas do negócio, como quantidade vendida, receita e custo, enquanto as dimensões fornecem o contexto descritivo dessas métricas, como tempo, produto e cliente (Kimball; Ross, 2013). Segundo Moraes, Silva e Antunes (2024), essa abordagem favorece a construção de indicadores e a tomada de decisão nas organizações.

O padrão mais utilizado é o Star Schema (Modelo Estrela), no qual uma tabela fato central é conectada diretamente a múltiplas tabelas de dimensão por chaves estrangeiras, minimizando o número de joins nas consultas analíticas (Databricks, 2024). Uma variação é o Snowflake Schema, que normaliza as dimensões em subdimensões, reduzindo redundância às custas de maior complexidade nas consultas (Inmon, 2002). Santana (2023) destaca que, mesmo com o avanço de plataformas de nuvem, o Star Schema permanece relevante por facilitar a integração de dados e a interpretação de KPIs e relatórios.

Um conceito central na modelagem dimensional é o de Slowly Changing Dimensions (SCD), que define como tratar mudanças nos atributos das dimensões ao longo do tempo. O Tipo 1 sobrescreve o valor antigo quando o histórico não tem relevância analítica, enquanto o Tipo 2, adotado neste trabalho para as dimensões de produto, cliente e vendedor, preserva o histórico inserindo um novo registro a cada alteração, com datas de vigência e indicador de registro atual (Hossain, 2025). Essa estratégia garante que consultas históricas recuperem os atributos

vigentes no momento exato de cada transação, o que constitui a essência da análise temporal em Data Warehouses (Thoughtspot, 2025).

2.2 ETL - EXTRAÇÃO, TRANSFORMAÇÃO E CARGA

O processo de ETL (Extract, Transform, Load) é responsável por integrar dados de fontes operacionais ao Data Warehouse. A fase de extração obtém os dados do sistema fonte, a transformação aplica regras de negócio, padronização e limpeza, e a carga persiste os dados transformados no DW (Kimball; Caserta, 2004). Segundo Khan et al. (2024), o ETL permanece um componente crítico em arquiteturas modernas de dados, sendo sua eficiência determinante para a qualidade e a confiabilidade das análises geradas.

A ETL incremental, também denominada carga delta, representa uma evolução em relação à carga completa. Em vez de reprocessar todos os dados a cada execução, identifica e processa apenas os registros inseridos ou modificados desde a última carga, reduzindo significativamente o tempo de execução e o consumo de recursos computacionais (Mwangi, 2025). Essa abordagem é especialmente relevante em ambientes com grandes volumes de dados e janelas de processamento reduzidas, sendo adotada por pipelines modernos como prática padrão de engenharia de dados (Integrate.Io, 2026).

As principais estratégias para identificação de registros delta são a comparação por timestamp, com base em campos como ModifiedDate presentes na maioria dos sistemas modernos, o uso de Change Data Capture (CDC), mecanismo que registra alterações em nível de log do banco de dados, e a comparação por hash dos atributos monitorados. Neste trabalho, adota-se a estratégia de timestamp, aproveitando o campo ModifiedDate presente em todas as tabelas relevantes do AdventureWorks.

3 METODOLOGIA DA PESQUISA

Este trabalho caracteriza-se como uma pesquisa aplicada de natureza descritiva, orientada ao desenvolvimento de uma solução de Business Intelligence para o domínio de vendas. A metodologia estrutura-se em quatro etapas sequenciais.

A primeira etapa consistiu na análise exploratória do modelo OLTP do AdventureWorks, com levantamento das tabelas relevantes do schema Sales, identificação dos relacionamentos entre entidades e compreensão das regras de negócio implícitas no modelo relacional. Essa fase subsidiou as decisões de modelagem dimensional adotadas nas etapas seguintes.

A segunda etapa envolveu a proposição do modelo multidimensional. Com base nos requisitos analíticos levantados, foram definidos os dez KPIs de vendas, a granularidade da tabela fato e as seis dimensões do modelo estrela. A escolha pelo Star Schema e pela estratégia de SCD Tipo 2 nas dimensões de maior variação foi fundamentada nos critérios de desempenho analítico e preservação histórica descritos no referencial teórico.

A terceira etapa compreendeu a implementação técnica do Data Warehouse. O schema do DW foi criado no SQL Server, incluindo as tabelas dimensionais, a tabela fato, os índices de performance e a tabela de controle ETL. Em paralelo, foi desenvolvido o pipeline de ETL incremental em Python 3.10+, utilizando a biblioteca pyodbc para conexão com o SQL Server, com suporte a carga delta por timestamp, SCD Tipo 2 e upsert na tabela fato.

A quarta etapa consistiu na validação dos indicadores por meio de consultas SQL analíticas executadas diretamente sobre o modelo estrela implementado, verificando a consistência dos resultados e a aderência de cada KPI à sua definição formal estabelecida na Tabela 1.

4 RESULTADOS E DISCUSSÃO

4.1 ANÁLISE DO MODELO OLTP – ADVENTUREWORKS

O AdventureWorks2016 é estruturado em schemas funcionais: Sales (vendas), Production (produção), Purchasing (compras), HumanResources (RH) e Person (cadastros de pessoas e endereços). Para este trabalho, o foco recaiu sobre o schema Sales, de onde se extraíram as principais entidades.

As tabelas centrais do módulo de vendas incluem: SalesOrderHeader (cabeçalho do pedido, com dados de cliente, vendedor, território, data e valores totais), SalesOrderDetail (linhas do pedido, com produto, quantidade e preço), Customer (clientes, podendo ser pessoas físicas ou lojas), SalesPerson (vendedores e suas cotas), SalesTerritory (territórios de venda) e SpecialOffer (promoções e descontos). O modelo relacional é altamente normalizado, com mais de 70 tabelas distribuídas nos schemas, o que exigiu um trabalho cuidadoso de desnormalização para a construção do modelo dimensional.

4.2 INDICADORES DE DESEMPENHO

Com base na análise do modelo OLTP e nos requisitos analíticos típicos de uma área de vendas, foram definidos dez indicadores de desempenho. A Tabela 1 apresenta o resumo dos KPIs propostos.

Tabela 1 - Indicadores de Desempenho (KPIs) propostos

N°	Nome do KPI	Descrição	Fórmula/ Lógica	Tabelas Envolvidas
01	Receita Total por Período	Soma da receita líquida por ano e mês	SUM (receita_liquida)	fato_vendas, dim_tempo
02	Ticket Médio por Pedido	Valor médio de cada pedido de venda	SUM (receita_liquida) / COUNT (DISTINCT nk_pedido)	fato_vendas, dim_tempo
03	Margem de Lucro Bruto	Percentual de margem bruta por categoria	SUM (lucro_bruto) / SUM (receita_liquida) * 100	fato_vendas, dim_produto
04	Top 10 Produtos por Receita	Ranking dos produtos mais rentáveis	RANK () OVER (ORDER BY SUM (receita_liquida) DESC)	fato_vendas, dim_produto

05	Desempenho vs. Cota	Percentual de atingimento da cota do vendedor	SUM (receita_liquida) / cota_anual * 100	fato_vendas, dim_vendedor
06	Receita por Território	Distribuição geográfica das vendas	SUM (receita_liquida) GROUP BY territorio	fato_vendas, dim_territorio
07	Impacto de Promoções	Desconto concedido por tipo de promoção	SUM (receita_bruta - receita_liquida)	fato_vendas, dim_promocao
08	Crescimento YoY	Variação de receita ano a ano	(receita_ano - receita_ano_ant) / receita_ano_ant	fato_vendas, dim_tempo
09	Análise RFM de Clientes	Segmentação por recência, frequência e valor monetário	RFM score por cliente	fato_vendas, dim_cliente
10	Sazonalidade por Trimestre	Índice sazonal por trimestre e ano	receita_tri / AVG (receita_tri) * 100	fato_vendas, dim_tempo

Fonte: Elaborado pelos autores (2026).

4.3 MODELO PROPOSTO

O modelo estrela foi projetado com uma tabela fato central (fato_vendas) conectada a seis dimensões: dim_tempo, dim_produto, dim_cliente, dim_vendedor, dim_territorio e dim_promocao. Essa estrutura suporta todos os dez KPIs definidos na seção anterior. A Figura 1 apresenta o diagrama do modelo estrela proposto, elaborado com a ferramenta Draw.io e disponível também no repositório GitHub do projeto. A tabela fato_vendas tem granularidade de linha de pedido (SalesOrderDetailID), o que permite análises tanto no nível de item quanto no nível de pedido consolidado. As métricas armazenadas incluem quantidade, preço unitário, desconto, receita bruta, receita líquida, custo total, lucro bruto, margem percentual, frete rateado e imposto rateado.

As dimensões dim_produto, dim_cliente e dim_vendedor implementam o padrão SCD Tipo 2 para preservar o histórico de alterações nos atributos descritivos. Cada registro inclui os campos data_inicio, data_fim e registro_atual, permitindo que consultas históricas recuperem o estado dos atributos quando a venda ocorreu.

4.4 DICIONARIO DE DADOS DO DATA WAREHOUSE

A Tabela 2 apresenta o dicionário de dados da tabela fato_vendas, principal componente do DW.

Tabela 2 - Dicionário de Dados: fato_vendas

Coluna	Tipo	Descrição	Origem (OLTP)
sk_venda	INT IDENTITY	Surrogate key da fato (PK)	Gerada pelo DW
nk_pedido	INT	Chave natural do pedido (SalesOrderID)	Sales.SalesOrderHeader
nk_detalhe	INT	Chave natural da linha do pedido (UK)	Sales.SalesOrderDetail
sk_tempo	INT (FK)	Referência à dim_tempo	Calculada via OrderDate
Coluna	**Tipo**	**Descrição**	**Origem (OLTP)**
sk_produto	INT (FK)	Referência à dim_produto (registro atual)	Sales.SalesOrderDetail
sk_cliente	INT (FK)	Referência à dim_cliente	Sales.SalesOrderHeader
sk_vendedor	INT (FK)	Referência à dim_vendedor (nullable)	Sales.SalesOrderHeader
sk_territorio	INT (FK)	Referência à dim_territorio	Sales.SalesOrderHeader
sk_promocao	INT (FK)	Referência à dim_promocao	Sales.SalesOrderDetail
quantidade	INT	Quantidade de itens vendidos	SalesOrderDetail.OrderQty
preco_unitario	DECIMAL (19,4)	Preço unitário sem desconto	SalesOrderDetail.UnitPrice
desconto_unitario	DECIMAL (19,4)	Desconto por unidade	SalesOrderDetail.UnitPriceDiscount
custo_padrao	DECIMAL (19,4)	Custo padrão do produto no OLTP	Production.Product.StandardCost
receita_bruta	DECIMAL (19,4)	qtd × preco_unitario	Calculada pelo ETL
receita_liquida	DECIMAL (19,4)	LineTotal já com desconto aplicado	SalesOrderDetail.LineTotal
custo_total	DECIMAL (19,4)	qtd × custo_padrao	Calculada pelo ETL
lucro_bruto	DECIMAL (19,4)	receita_liquida - custo_total	Calculada pelo ETL
margem_percentual	DECIMAL (10,4)	lucro_bruto / receita_liquida	Calculada pelo ETL
frete_rateado	DECIMAL (19,4)	Frete proporcional à linha do pedido	Rateio pelo ETL
imposto_rateado	DECIMAL (19,4)	Imposto proporcional à linha do pedido	Rateio pelo ETL
data_carga	DATETIME2	Data e hora da inserção no DW	ETL
data_atualizacao	DATETIME2	Data e hora da última atualização	ETL

Fonte: Elaborado pelos autores (2026).

4.5 ESTRATÉGIA DE ETL INCREMENTAL

A ETL foi desenvolvida em Python 3.10+ utilizando a biblioteca pyodbc para conexão com o SQL Server. A estratégia incremental baseia-se na comparação do campo ModifiedDate de cada tabela OLTP com a última data de referência armazenada na tabela de controle etl_controle do DW. A cada execução, apenas os registros cujo ModifiedDate é maior que esse valor são extraídos e processados.

O fluxo de execução segue a sequência: conexão com os bancos OLTP (AdventureWorks) e DW; carga das dimensões com baixa variação (dim_tempo, dim_territorio, dim_promocao); carga das dimensões com SCD Tipo 2 (dim_produto, dim_cliente, dim_vendedor); carga incremental da tabela fato_vendas com cursor server-side para grandes volumes; atualização dos registros de controle com o timestamp do último registro processado; e commit em lotes de 500 registros (BATCH_SIZE configurável).

Para as dimensões SCD Tipo 2, o processo consiste em fechar o registro vigente, definindo data_fim como a data atual e registro_atual como 0, e inserir um novo registro com os dados atualizados e registro_atual = 1. Dessa forma, o histórico completo de alterações nos atributos dimensionais é preservado, permitindo análises históricas precisas.

A tabela fato_vendas utiliza lógica de upsert, verificando a existência do nk_detalhe antes de inserir ou atualizar, garantindo que registros já carregados sejam atualizados caso tenham sido modificados no OLTP. O campo nk_detalhe (SalesOrderDetailID) serve como chave natural única para controle de duplicidade. Um cache em memória de lookups de surrogate keys é mantido durante a execução para evitar consultas repetidas ao DW a cada linha processada, reduzindo significativamente o número de round-trips ao banco de dados.

4.6 JUSTIFICATIVA DAS DECISÕES DE MODELAGEM

A escolha pelo Star Schema em detrimento do Snowflake Schema justifica-se pela prioridade ao desempenho de consultas analíticas. No modelo estrela, cada KPI pode ser calculado com um único join entre a fato e a dimensão relevante, sem necessidade de joins em cadeia. Embora o modelo estrela apresente alguma redundância nos atributos das dimensões, essa característica é aceitável, e até desejável em um DW voltado para análise, pois simplifica as queries e melhora a legibilidade.

A granularidade da tabela fato ao nível de linha de pedido (SalesOrderDetailID) foi escolhida por oferecer máxima flexibilidade analítica. Com essa granularidade, é possível tanto analisar itens individuais quanto agregar por pedido, por cliente, por período ou por qualquer combinação de dimensões. Uma granularidade mais alta (nível de pedido) teria implicado em perda de informação sobre a composição de cada venda.

A decisão de implementar SCD Tipo 2 nas dimensões de produto, cliente e vendedor baseia-se na necessidade de preservar o histórico de atributos que podem mudar ao longo do tempo. Por exemplo, o preço de lista de um produto ou o território de um vendedor podem ser alterados, e é fundamental que os relatórios históricos reflitam os valores vigentes no momento de cada venda, não os valores atuais. Para as dimensões de território e promoção, adotou-se SCD Tipo 1 (sobrescrita), pois alterações nesses atributos não têm relevância histórica para os KPIs definidos.

4.7 REPOSITÓRIO GITHUB E VALIDAÇÃO DA IMPLEMENTAÇÃO

A fim de comprovar a implementação prática do Data Warehouse, o projeto foi executado em ambiente SQL Server com validação em todas as etapas do processo. As Figuras 2, 3, 4 e 5 registram as principais etapas da execução.

A Figura 2 apresenta a configuração da conexão ODBC entre o ambiente Python e as bases de dados do SQL Server, etapa fundamental para que o pipeline de ETL incremental pudesse se comunicar com os bancos OLTP e DW durante a execução.

A Figura 3 ilustra a execução do script de criação do Data Warehouse, com a geração das tabelas dimensionais, da tabela fato_vendas, dos índices de performance e da tabela de controle etl_controle, confirmando que o schema foi criado sem erros no ambiente SQL Server.

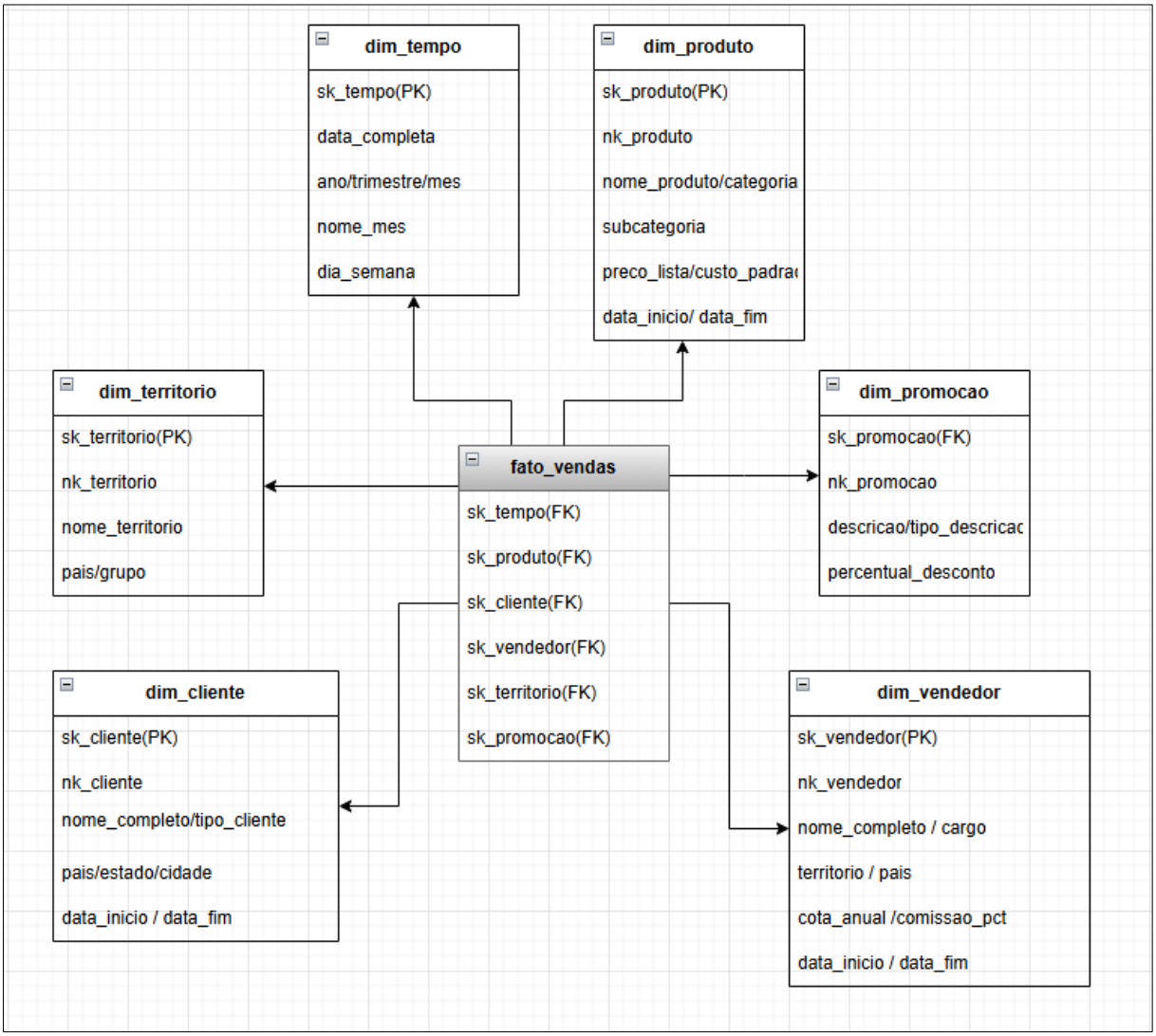
A Figura 4 demonstra a execução do módulo Python de ETL incremental, com o carregamento das dimensões e da tabela fato a partir dos dados do AdventureWorks, evidenciando o funcionamento da estratégia de carga delta baseada no campo ModifiedDate.

A Figura 5 exibe a execução de uma das consultas SQL de validação dos KPIs diretamente sobre o modelo estrela implementado, confirmando que o modelo dimensional suporta as análises propostas e retorna os resultados esperados.

Todo o projeto está disponível no repositório GitHub dos integrantes do grupo, contendo: o script de criação do DW (dw_schema.sql), os scripts SQL dos dez KPIs (kpis_dw.sql) e o módulo Python de ETL incremental (etl_incremental.py).

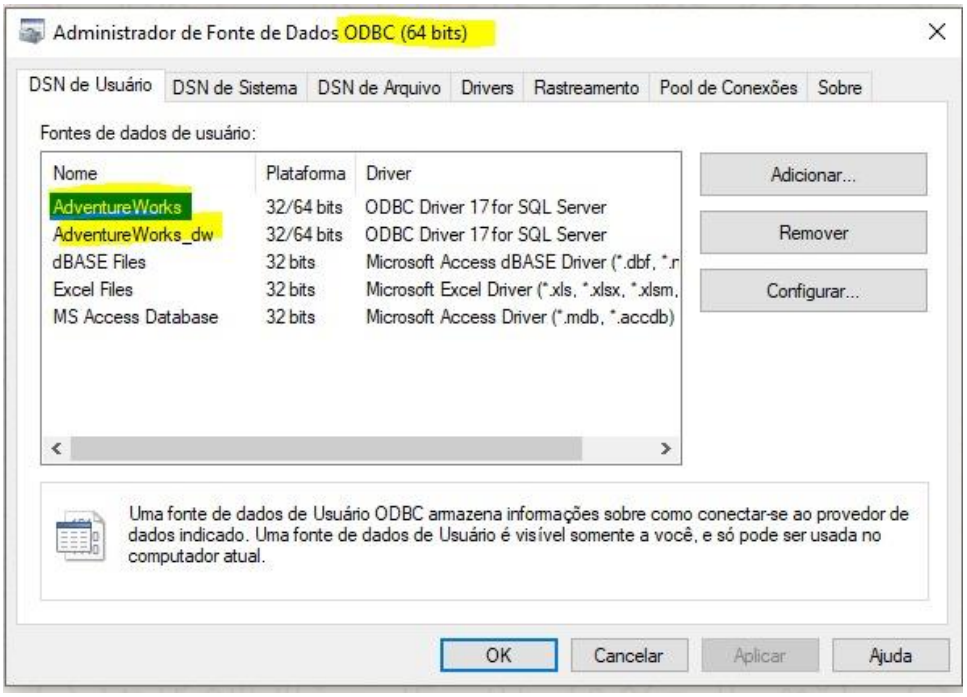
https://github.com/PereiraRicardo/OLAP-ETL, https://github.com/Caac1984
https://github.com/VitinTurial, https://github.com/hENRIQUEBRUNELI/OLAP-ETL_1

Figura 1 - Diagrama do Modelo Estrela Proposto.



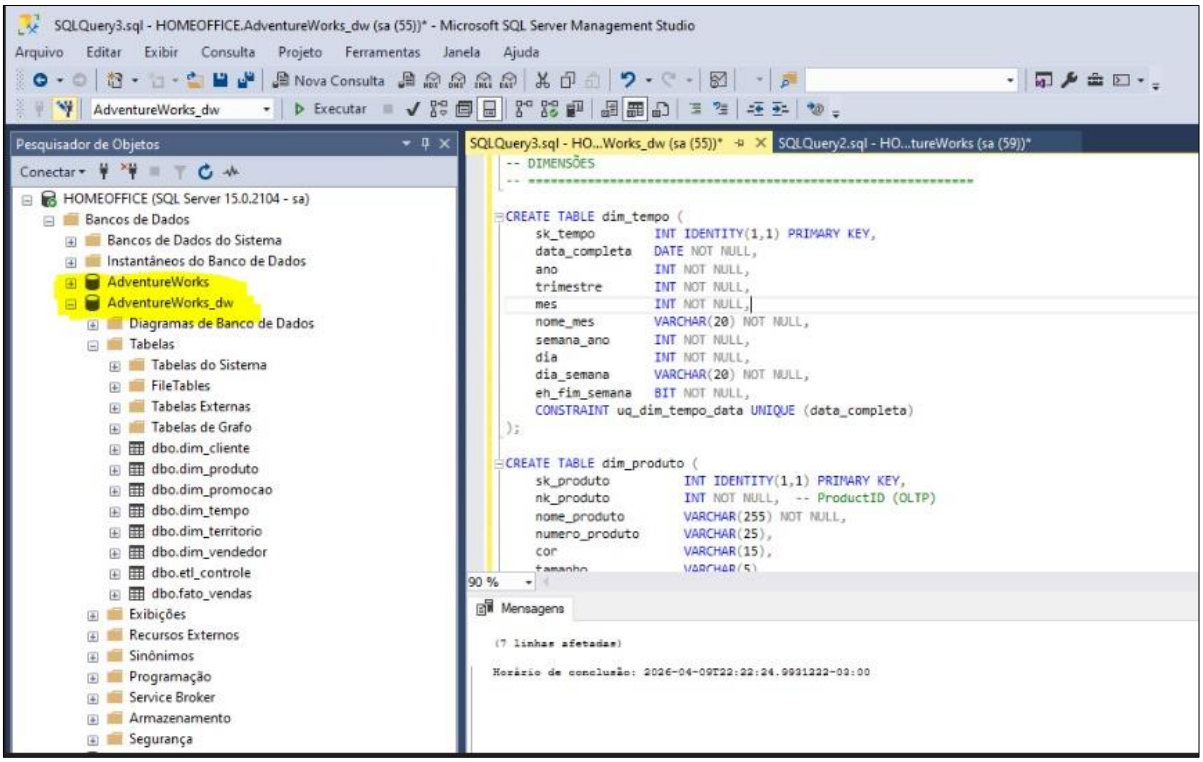
Fonte: Elaborado pelos autores (2026)

Figura 2 - Configuração da conexão ODBC entre Python e SQL Server.



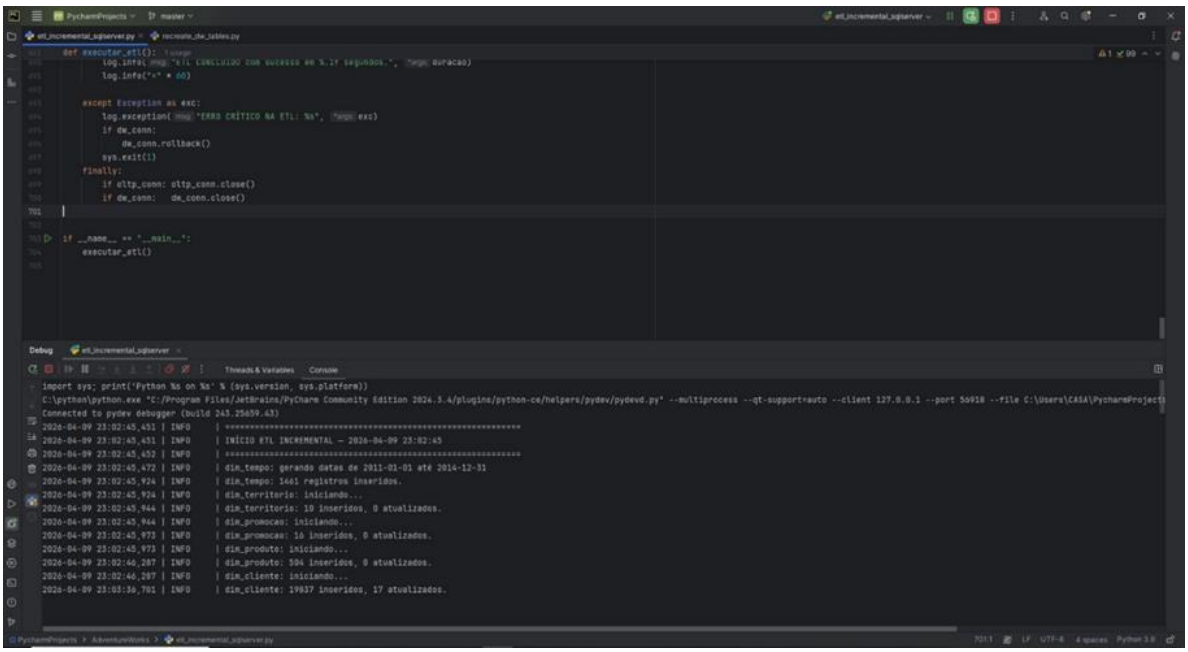
Fonte: Elaborado pelos autores (2026).

Figura 3 - Execução do script de criação do Data Warehouse no SQL Server.



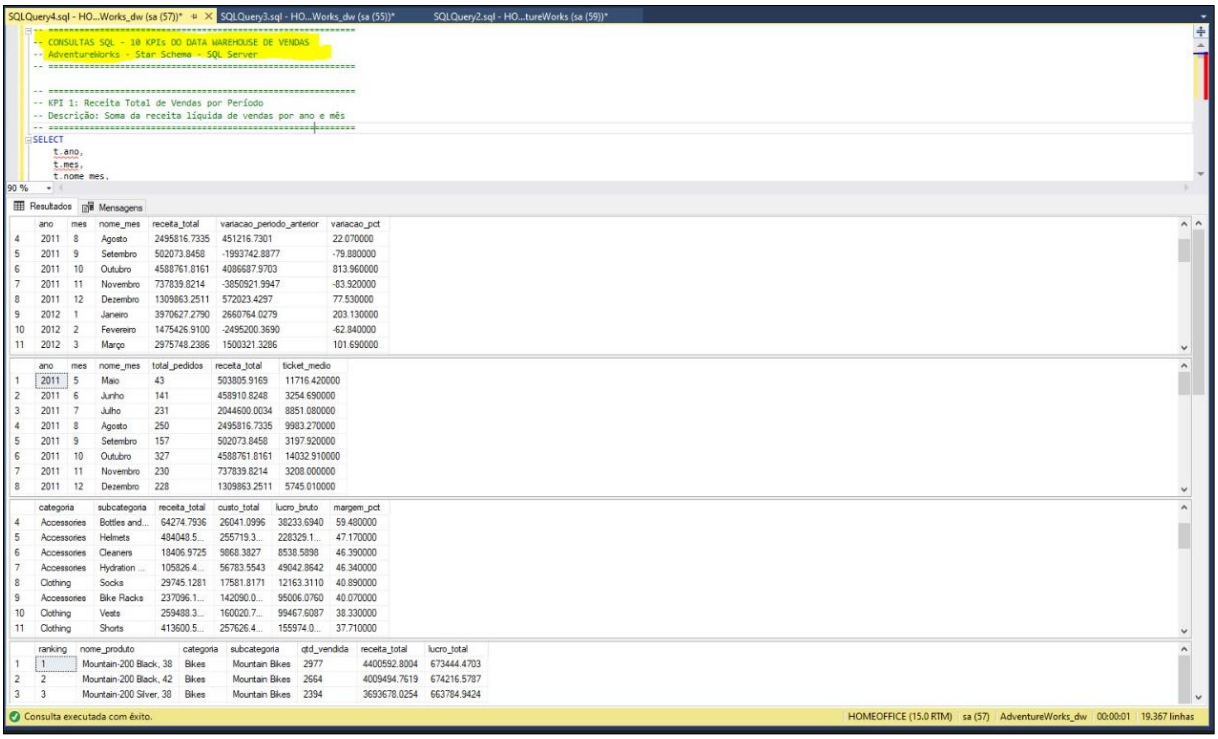
Fonte: Elaborado pelos autores (2026).

Figura 4 - Execução do pipeline de ETL incremental em Python.



Fonte: Elaborado pelos autores (2026).

Figura 5 - Execução de consulta SQL de validação dos KPIs.



Fonte: Elaborado pelos autores (2026).

Exemplo de consulta SQL para o KPIs:

```
-- ==============================================================
-- KPI 1: Receita Total de Vendas por Período
-- Descrição: Soma da receita líquida de vendas por ano e mês
-- ==============================================================
SELECT
    t.ano,
    t.mes,
    t.nome_mes,
    SUM(f.receita_liquida)                    AS receita_total,
    SUM(f.receita_liquida) - LAG(SUM(f.receita_liquida))
        OVER (ORDER BY t.ano, t.mes)          AS variacao_periodo_anterior,
    ROUND(
        (SUM(f.receita_liquida) - LAG(SUM(f.receita_liquida))
            OVER (ORDER BY t.ano, t.mes))
        / NULLIF(LAG(SUM(f.receita_liquida))
            OVER (ORDER BY t.ano, t.mes), 0) * 100, 2
    )                                         AS variacao_pct
FROM fato_vendas f
JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
GROUP BY t.ano, t.mes, t.nome_mes
ORDER BY t.ano, t.mes;
-- ==============================================================
-- KPI 2: Ticket Médio por Pedido
-- Descrição: Valor médio de cada pedido de venda
-- ==============================================================
SELECT
    t.ano,
    t.mes,
    t.nome_mes,
    COUNT(DISTINCT f.nk_pedido)               AS total_pedidos,
    SUM(f.receita_liquida)                    AS receita_total,
    ROUND(SUM(f.receita_liquida)
        / NULLIF(COUNT(DISTINCT f.nk_pedido), 0), 2) AS ticket_medio
FROM fato_vendas f
```

```
JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
GROUP BY t.ano, t.mes, t.nome_mes
ORDER BY t.ano, t.mes;
-- ==============================================================
-- KPI 3: Margem de Lucro Bruto por Categoria de Produto
-- Descrição: Percentual de margem bruta agrupado por categoria
-- ==============================================================
SELECT
   p.categoria,
   p.subcategoria,
   SUM(f.receita_liquida)                    AS receita_total,
   SUM(f.custo_total)                        AS custo_total,
   SUM(f.lucro_bruto)                        AS lucro_bruto,
   ROUND(SUM(f.lucro_bruto)
      / NULLIF(SUM(f.receita_liquida), 0) * 100, 2) AS margem_pct
FROM fato_vendas f
JOIN dim_produto p ON f.sk_produto = p.sk_produto
WHERE p.registro_atual = 1                   -- BIT: TRUE -> 1
GROUP BY p.categoria, p.subcategoria
ORDER BY margem_pct DESC;
-- ==============================================================
-- KPI 4: Ranking de Produtos Mais Vendidos (por Receita)
-- Descrição: Top 10 produtos por receita líquida
-- ==============================================================
SELECT TOP 10
   RANK() OVER (ORDER BY SUM(f.receita_liquida) DESC) AS ranking,
   p.nome_produto,
   p.categoria,
   p.subcategoria,
   SUM(f.quantidade)                         AS qtd_vendida,
   SUM(f.receita_liquida)                    AS receita_total,
   SUM(f.lucro_bruto)                        AS lucro_total
FROM fato_vendas f
JOIN dim_produto p ON f.sk_produto = p.sk_produto
WHERE p.registro_atual = 1
```

```
GROUP BY p.nome_produto, p.categoria, p.subcategoria
ORDER BY receita_total DESC;
-- ==============================================================
-- KPI 5: Desempenho de Vendedores vs. Cota
-- Descrição: Comparativo entre receita realizada e cota anual
-- ==============================================================
SELECT
   v.nome_completo                          AS vendedor,
   v.territorio,
   t.ano,
   SUM(f.receita_liquida)                   AS receita_realizada,
   MAX(v.cota_anual)                        AS cota_anual,
   ROUND(SUM(f.receita_liquida)
      / NULLIF(MAX(v.cota_anual), 0) * 100, 2)   AS pct_atingimento_cota,
   CASE
      WHEN SUM(f.receita_liquida) >= MAX(v.cota_anual) THEN 'Atingiu'
      WHEN SUM(f.receita_liquida) >= MAX(v.cota_anual) * 0.8 THEN 'Próximo'
      ELSE 'Abaixo'
   END                                      AS status_cota
FROM fato_vendas f
JOIN dim_vendedor v ON f.sk_vendedor = v.sk_vendedor
JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
WHERE v.registro_atual = 1
GROUP BY v.nome_completo, v.territorio, t.ano
ORDER BY t.ano, pct_atingimento_cota DESC;
-- ==============================================================
-- KPI 6: Receita por Território / Região
-- Descrição: Distribuição geográfica das vendas
-- ==============================================================
SELECT
   ter.grupo                                AS regiao_grupo,
   ter.pais,
   ter.nome_territorio,
   t.ano,
   COUNT(DISTINCT f.nk_pedido)              AS total_pedidos,
```

```
    SUM(f.quantidade)                    AS qtd_itens,
    SUM(f.receita_liquida)               AS receita_total,
    ROUND(SUM(f.receita_liquida)
        / SUM(SUM(f.receita_liquida)) OVER (PARTITION BY t.ano) * 100, 2) AS
pct_receita_ano
FROM fato_vendas f
JOIN dim_territorio ter ON f.sk_territorio = ter.sk_territorio
JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
GROUP BY ter.grupo, ter.pais, ter.nome_territorio, t.ano
ORDER BY t.ano, receita_total DESC;
-- ================================================================
-- KPI 7: Taxa de Desconto e Impacto na Receita
-- Descrição: Análise do impacto das promoções na receita
-- ================================================================
SELECT
    pr.descricao                         AS promocao,
    pr.tipo_desconto,
    pr.categoria,
    ROUND(AVG(pr.percentual_desconto) * 100, 2)    AS desconto_medio_pct,
    COUNT(DISTINCT f.nk_pedido)                    AS pedidos_com_desconto,
    SUM(f.quantidade)                    AS qtd_vendida,
    SUM(f.receita_bruta)                 AS receita_bruta_total,
    SUM(f.receita_liquida)               AS receita_liquida_total,
    SUM(f.receita_bruta - f.receita_liquida)       AS total_desconto_concedido
FROM fato_vendas f
JOIN dim_promocao pr ON f.sk_promocao = pr.sk_promocao
GROUP BY pr.descricao, pr.tipo_desconto, pr.categoria, pr.percentual_desconto
ORDER BY total_desconto_concedido DESC;
-- ================================================================
-- KPI 8: Crescimento de Receita Ano a Ano (YoY)
-- Descrição: Comparativo de receita entre anos consecutivos
-- ================================================================
WITH receita_anual AS (
    SELECT
        t.ano,
```

```
        SUM(f.receita_liquida)  AS receita_total
    FROM fato_vendas f
    JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
    GROUP BY t.ano
)
SELECT
    ano,
    receita_total,
    LAG(receita_total) OVER (ORDER BY ano)          AS receita_ano_anterior,
    receita_total - LAG(receita_total) OVER (ORDER BY ano) AS variacao_absoluta,
    ROUND(
        (receita_total - LAG(receita_total) OVER (ORDER BY ano))
        / NULLIF(LAG(receita_total) OVER (ORDER BY ano), 0) * 100, 2
    )                                       AS crescimento_yoy_pct
FROM receita_anual
ORDER BY ano;
-- ==============================================================
-- KPI 9: Frequência de Compra por Cliente (RFM Simplificado)
-- Descrição: Recência, Frequência e Valor Monetário dos clientes
-- ==============================================================
WITH rfm AS (
    SELECT
        f.sk_cliente,
        MAX(t.data_completa)                    AS ultima_compra,
        COUNT(DISTINCT f.nk_pedido)             AS frequencia,
        SUM(f.receita_liquida)                  AS valor_total,
        DATEDIFF(DAY, MAX(t.data_completa), CAST(GETDATE() AS DATE))
                                                AS dias_desde_ultima_compra
        -- CURRENT_DATE -> CAST(GETDATE() AS DATE)
        -- subtração de datas -> DATEDIFF(DAY, ...)
    FROM fato_vendas f
    JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
    GROUP BY f.sk_cliente
)
SELECT
```

```
    c.nome_completo,
    c.tipo_cliente,
    c.pais,
    rfm.ultima_compra,
    rfm.dias_desde_ultima_compra                AS recencia_dias,
    rfm.frequencia                              AS total_pedidos,
    ROUND(rfm.valor_total, 2)                   AS valor_monetario,
    ROUND(rfm.valor_total / rfm.frequencia, 2)  AS ticket_medio,
    CASE
        WHEN rfm.dias_desde_ultima_compra <= 365
            AND rfm.frequencia >= 5
            AND rfm.valor_total >= 10000        THEN 'Premium'
        WHEN rfm.dias_desde_ultima_compra <= 730
            AND rfm.frequencia >= 2             THEN 'Recorrente'
        ELSE 'Ocasional'
    END                                         AS segmento_cliente
FROM rfm
JOIN dim_cliente c ON rfm.sk_cliente = c.sk_cliente
WHERE c.registro_atual = 1
ORDER BY rfm.valor_total DESC;
-- ==============================================================
-- KPI 10: Sazonalidade de Vendas por Trimestre
-- Descrição: Padrão sazonal das vendas ao longo dos trimestres
-- ==============================================================
SELECT
    t.ano,
    t.trimestre,
    CONCAT('Q', t.trimestre, '/', t.ano)        AS periodo,
    COUNT(DISTINCT f.nk_pedido)                 AS total_pedidos,
    SUM(f.quantidade)                           AS qtd_itens,
    SUM(f.receita_liquida)                      AS receita_total,
    ROUND(AVG(f.receita_liquida), 2)            AS receita_media_por_item,
    ROUND(
        SUM(f.receita_liquida)
        / SUM(SUM(f.receita_liquida)) OVER (PARTITION BY t.ano) * 100, 2
```

```
  )                                   AS pct_receita_no_ano,
  ROUND(
    SUM(f.receita_liquida)
    / NULLIF(
      AVG(SUM(f.receita_liquida)) OVER (PARTITION BY t.trimestre), 0
    ) * 100, 2
  )                                   AS indice_sazonalidade
FROM fato_vendas f
JOIN dim_tempo t ON f.sk_tempo = t.sk_tempo
GROUP BY t.ano, t.trimestre
ORDER BY t.ano, t.trimestre;
```

5 CONSIDERAÇÕES FINAIS

Este trabalho demonstrou a viabilidade da modelagem multidimensional com padrão Star Schema para a construção de um Data Warehouse voltado à área de vendas. O modelo proposto, com uma tabela fato e seis dimensões, mostrou-se adequado para suportar todos os dez KPIs definidos, e os scripts SQL desenvolvidos comprovaram o funcionamento das consultas analíticas sobre o modelo implementado.

O pipeline de ETL incremental desenvolvido em Python apresentou comportamento eficiente, processando apenas os registros alterados a cada execução e mantendo a consistência dos dados carregados no Data Warehouse. O projeto foi disponibilizado integralmente no repositório GitHub do grupo, contendo os scripts de criação do banco, os KPIs e o módulo de ETL.

Como trabalhos futuros, sugere-se a expansão do modelo para outros módulos do AdventureWorks, a implementação de um orquestrador como Apache Airflow para automatizar a ETL e a criação de dashboards em ferramentas como Power BI para visualização dos indicadores.

REFERÊNCIAS

DATABRICKS. What is Star Schema? 2024. Disponível em: https://www.databricks.com/blog/what-is-star-schema. Acesso em: mar. 2026.

HOSSAIN, Manik. Designing Slowly Changing Dimensions (SCD Type 2) in Modern Data Warehouses. Medium, ago. 2025. Disponível em: https://medium.com/@manik.ruet08/designing-slowly-changing-dimensions-scd-type-2-in-modern-data-warehouses-0d627ff804d9. Acesso em: mar. 2026.

INMON, W. H. Building the Data Warehouse. 4. ed. Indianapolis: Wiley Publishing, 2002.

INTEGRATE.IO. ETL Frameworks in 2026 for Future-Proof Data Pipelines. 2026. Disponível em: https://www.integrate.io/blog/etl-frameworks-in-2025-designing-robust-future-proof-data-pipelines/. Acesso em: mar. 2026.

KHAN et al. An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing. ResearchGate, 2024. Disponível em: https://www.researchgate.net/publication/377602160. Acesso em: mar. 2026.

KIMBALL, R.; CASERTA, J. The Data Warehouse ETL Toolkit. Indianapolis: Wiley Publishing, 2004.

KIMBALL, R.; ROSS, M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. 3. ed. Indianapolis: John Wiley & Sons, 2013.

MICROSOFT. AdventureWorks sample databases. Disponível em: https://github.com/Microsoft/sql-server-samples. Acesso em: mar. 2026.

MORAES, Marcos Henrique de; SILVA, João Manoel da; ANTUNES, Joaquim M. F. Data Warehouse e usabilidade do modelo dimensional. Prospectus — Revista Científica da FATEC Itapira, Itapira, v. 6, n. 1, p. 415-445, jan./jun. 2024. Disponível em: https://prospectus.fatecitapira.edu.br. Acesso em: mar. 2026.

MWANGI, Francis. Building a Scalable ETL Pipeline: From MySQL to PostgreSQL with Python, Docker and Airflow. Medium, abr. 2025. Disponível em: https://medium.com/@franco313mwas/building-a-scalable-etl-pipeline-from-mysql-to-postgresql-with-python-docker-airflow-4200149a0757. Acesso em: mar. 2026.

SANTANA, Raissa Silva Menezes de. Modelagem de dados em Data Warehouses modernos: comparação entre modelos no contexto nas novas plataformas analíticas. Revista Científica Semana Acadêmica, Fortaleza, v. 11, n. 234, jun. 2023. Disponível em: https://semanaacademica.org.br/artigo/modelagem-de-dados-em-data-warehouses-modernos-comparacao-entre-modelos-no-contexto-nas-novas. Acesso em: mar. 2026.

THOUGHTSPOT. Slowly Changing Dimensions (SCD): 4 Types & How to Implement. 2025. Disponível em: https://www.thoughtspot.com/data-trends/data-modeling/slowly-changing-dimensions-in-data-warehouse. Acesso em: mar. 2026.